

1. FCFS (First Come First Served)

Idea

The process that **arrives first gets the CPU first**.

Type

- Non-Preemptive

Algorithm Skeleton

```
Sort processes by Arrival Time.  
  
For each process:  
    Execute it completely.  
    Move to the next process.
```

Advantages

- Very simple
- Easy to implement
- Low context switching

Disadvantages

- Convoy Effect
- Poor average waiting time
- Bad for interactive systems

Time Complexity

```
O(n log n) (sorting by arrival time)
```

Interview Points

- Non-preemptive
- Suffers from Convoy Effect
- Fair but inefficient

2. SJF (Shortest Job First)

Idea

Execute the process having the **smallest Burst Time** among the available processes.

Type

- Non-Preemptive

Algorithm Skeleton

```
While processes remain:  
  
    Select all arrived processes.  
  
    Choose process with minimum Burst Time.  
  
    Execute it completely.
```

Advantages

- Minimum average waiting time (optimal)
- Better throughput than FCFS

Disadvantages

- Burst time must be known or predicted
- Starvation possible

Time Complexity

```
0(n²)

or

0(n log n) using Priority Queue
```

Interview Points

- Non-preemptive
- Optimal average waiting time
- Can starve long jobs

3. SRTF (Shortest Remaining Time First)

Idea

Always execute the process having the **smallest remaining CPU time**.

If a shorter job arrives,

interrupt the current process.

Type

- Preemptive

Algorithm Skeleton

```
At every arrival or completion:

    Select process with
    minimum Remaining Time.

    If needed,
    preempt current process.
```

Advantages

- Very low average waiting time
- Better response time

Disadvantages

- Many context switches
- Starvation possible
- Burst prediction required

Time Complexity

```
0(n log n)

(using Priority Queue)
```

Interview Points

- Preemptive version of SJF
- Better response than SJF
- Higher overhead

4. Priority Scheduling

Idea

Execute the process having the **highest priority**.

Priority may be:

- Lowest number = highest priority
- or
- Highest number = highest priority

(depending on implementation)

Types

- Non-preemptive
- Preemptive

Algorithm Skeleton

```
Select arrived process
having highest priority.
```

```
If preemptive:
```

```
    If higher priority arrives,
    interrupt current process.
```

Advantages

- Important tasks execute first
- Good for real-time systems

Disadvantages

- Starvation
- Needs Aging

Time Complexity

```
O(n log n)
```

Interview Points

- Can be preemptive or non-preemptive
- Aging prevents starvation

5. Round Robin (RR)

Idea

Each process gets a fixed **Time Quantum**.

If unfinished,

move it to the end of the Ready Queue.

Type

- Preemptive

Algorithm Skeleton

```
Ready Queue
```

```
While queue not empty:
```

```
    Execute process
    for Time Quantum.
```

```
    If finished:
```

```
        Remove
```

```
    Else:
```

```
        Push to back
```

Advantages

- Fair
- Excellent response time
- Interactive systems

Disadvantages

- Too many context switches if quantum is small
- Behaves like FCFS if quantum is very large

Time Complexity

```
O(number of context switches)
```

Interview Points

Small Quantum

- Good response
- High overhead

Large Quantum

- Low overhead
- Approaches FCFS

6. Multilevel Queue (MLQ)

Idea

Processes are divided into **different queues**.

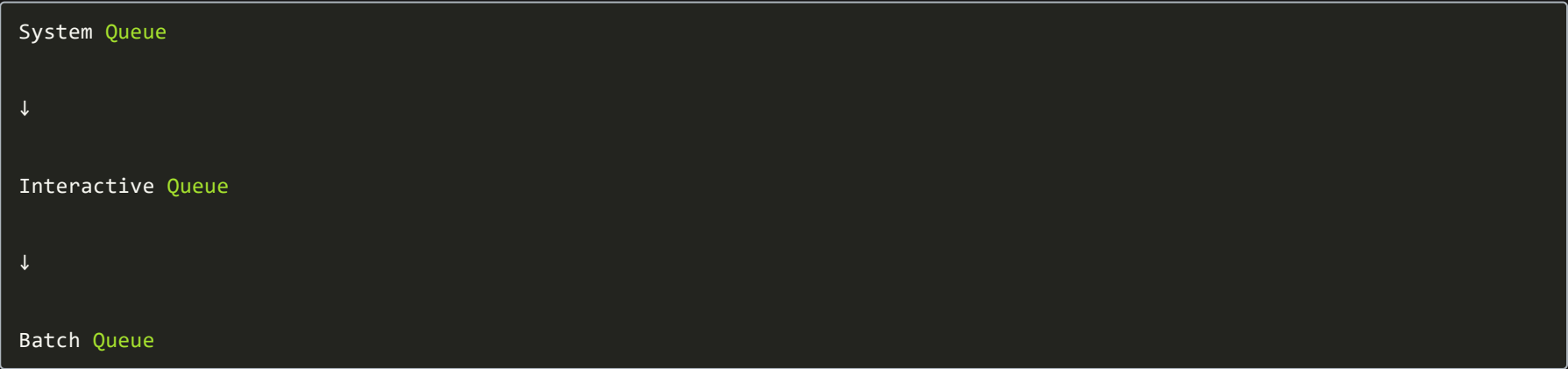
Each queue has its own scheduling algorithm.

Processes **cannot move** between queues.

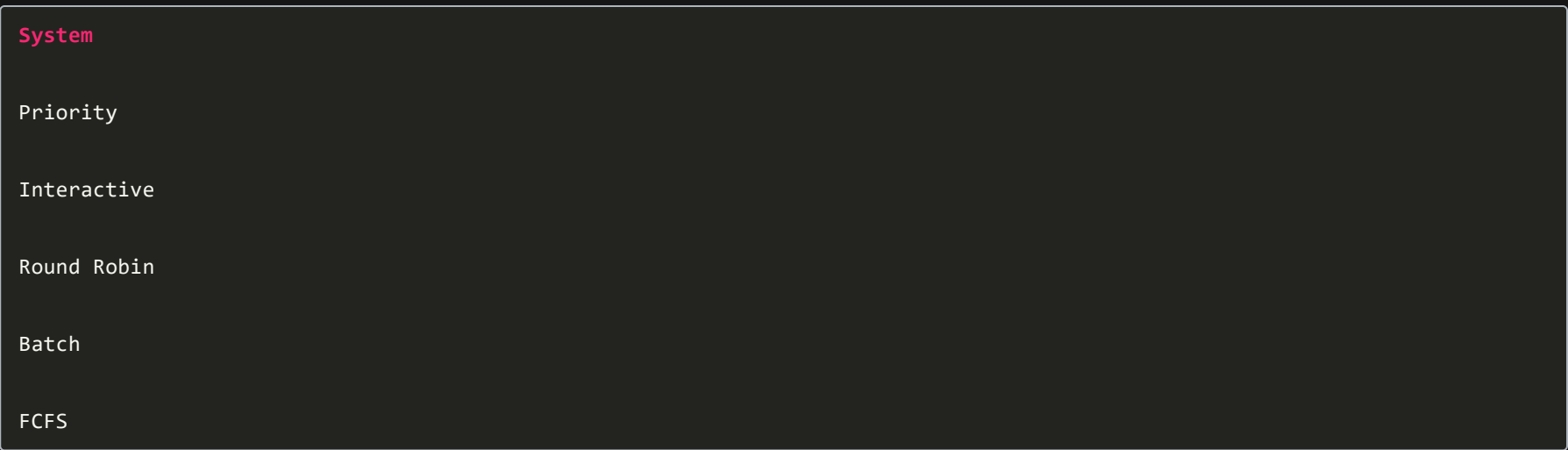
Type

Usually Preemptive between queues.

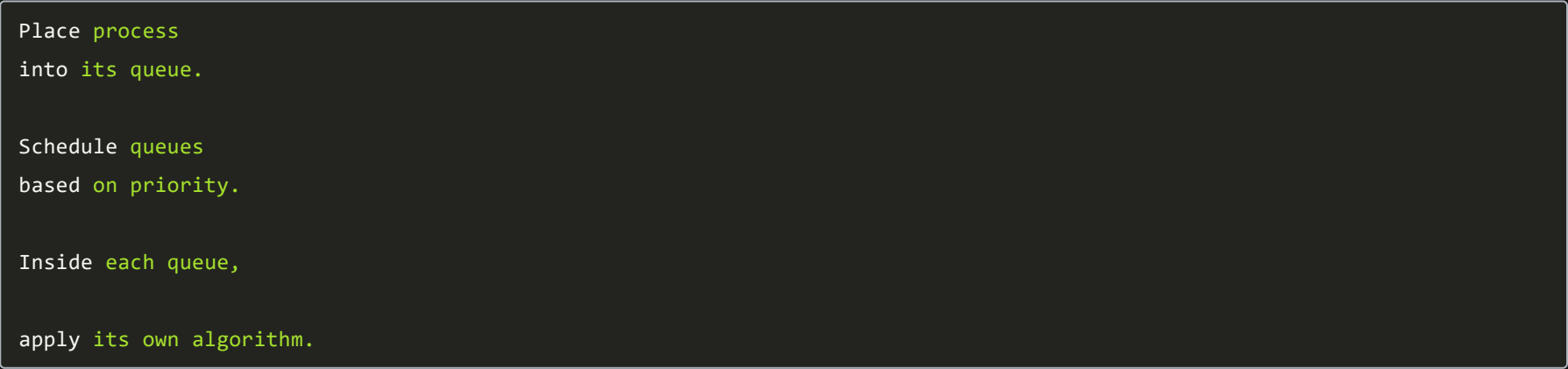
Example



Each queue may use



Algorithm Skeleton



Advantages

- Different scheduling for different process types
- Good organization

Disadvantages

- Starvation possible
- Fixed queue assignment

Interview Points

- No movement between queues

7. Multilevel Feedback Queue (MLFQ)

Idea

Like MLQ,

but processes **can move between queues**.

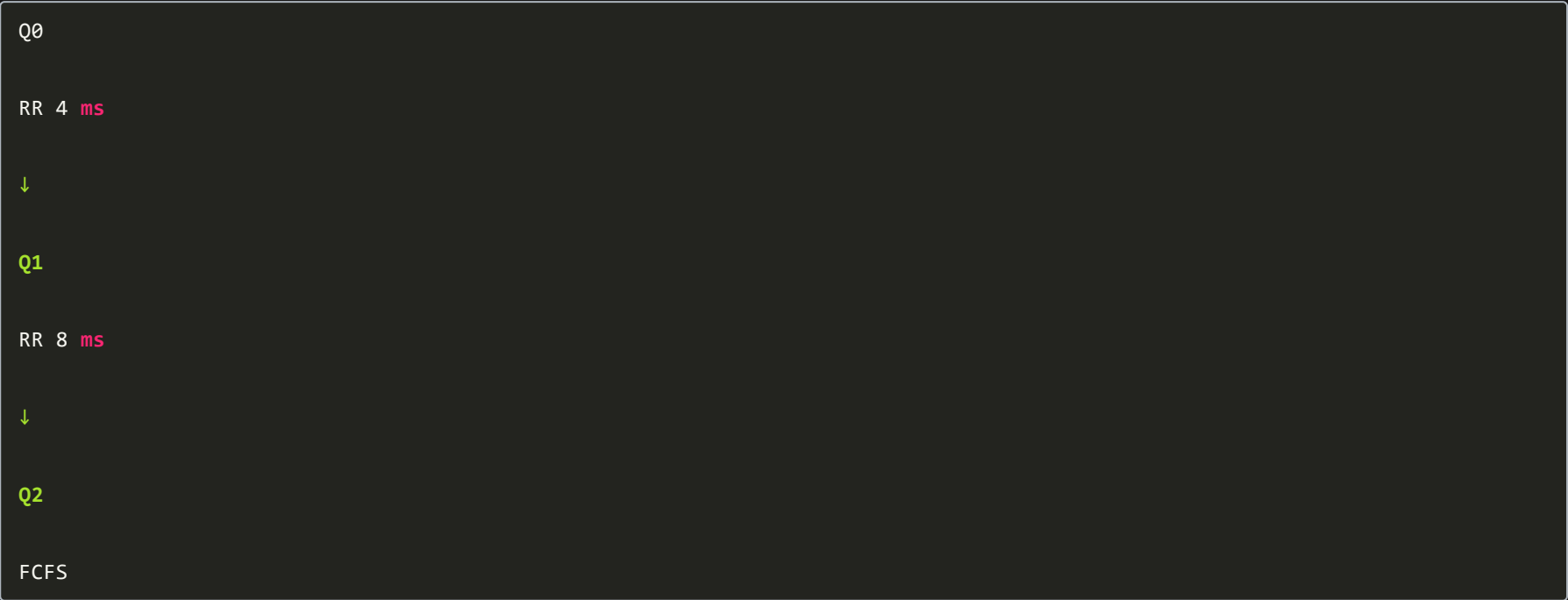
CPU-intensive processes gradually move to lower-priority queues.

Interactive processes stay in higher-priority queues.

Type

Preemptive

Example



If process uses entire quantum

↓

Move down.

If process waits for I/O

↓

Stay high.

Algorithm Skeleton



Advantages

- Excellent response time
- Prevents starvation
- Adapts automatically
- Used in real operating systems

Disadvantages

- Complex
- Difficult to tune

Interview Points

- Dynamic priorities
- Processes move between queues
- Common in modern OS schedulers

Comparison Table

Algorithm	Preemptive	Selection Basis	Starvation	Best For	
FCFS	✗	Arrival Time	✗	Batch jobs	
SJF	✗	Shortest Burst Time	✓	Minimum average waiting time	
SRTF	✓	Shortest Remaining Time	✓	Better response time	
Priority	Both	Highest Priority	✓	Real-time/critical tasks	
Round Robin	✓	Time Quantum	✗	Interactive systems	
MLQ	Usually	Queue Priority	✓	Different process classes	
MLFQ	✓	Dynamic Priority	Rare	General-purpose OS	

Interview Cheat Sheet

Algorithm	One-line Rule
FCFS	Run the process that arrived first.
SJF	Run the available process with the smallest burst time.
SRTF	Always run the process with the smallest remaining burst time; preempt if needed.
Priority	Run the highest-priority process; may be preemptive or non-preemptive.
Round Robin	Give each process a fixed time quantum in cyclic order.
MLQ	Separate processes into fixed queues; each queue has its own scheduling policy.
MLFQ	Like MLQ, but processes can move between queues based on their CPU usage and behavior.